

JAVA INTERVIEW PREPARATION

CHAPTER 61

SLOW API AND LATENCY
DIAGNOSIS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Diagnosing Slow APIs and Latency Regressions

Senior / Lead Java Developer Reading Chapter

A slow API is rarely diagnosed by reading one controller method. End-to-end latency includes routing, queueing, authentication, application execution, connection acquisition, database work, downstream calls, serialization, network transfer, and client consumption. The slowest component may be outside the Java process, while the Java service still amplifies it through retries or exhausted pools.

The investigation must establish where elapsed time was spent and why that path changed.

```
client -> edge -> server queue -> Java execution -> database/downstreams
      <- network <- serialization <- response construction <- results
```

Define the symptom precisely

"The API is slow" needs dimensions:

- which normalized endpoint and method;
- p50, p95, p99, maximum, and timeout rate;
- start time and whether it is continuous or periodic;
- affected tenant, region, instance, version, or payload band;
- request rate, success rate, and response size;
- client-observed versus server-observed duration.

Averages hide tail latency. If 99 requests complete in 20 ms and one takes 10 seconds, the average is about 120 ms and describes almost nobody. Percentiles need adequate sample count and a stated time window.

Compare with an SLO and a healthy baseline. A p99 regression from 200 ms to 600 ms may be serious even when the five-second client timeout is not reached.

Client time and server time are different

Server instrumentation often begins after the request reaches the application and ends when the response is handed to the server framework. The client includes DNS, connection establishment, TLS, proxies, network transfer, retries, and possibly waiting to read the body.

```
client observed: 1,200 ms
edge observed:   900 ms
Java server:    180 ms

missing time is evidence: routing, queueing, transfer, client, or retries
```

Align timestamps and correlation IDs across layers. Check whether a client library reports one logical operation containing several attempts while the server reports each attempt separately.

Clock skew complicates absolute timestamp subtraction between hosts. Distributed tracing normally uses local durations within spans and parent-child relationships, but broken propagation and sampling can still mislead.

Protect service while preserving evidence

For active impact, reduce optional work, pause bulk jobs, rate-limit abusive traffic, disable a regressed feature, or revert a known deployment. Do not increase every timeout: longer waits hold threads, connections, memory, and client resources, often making the queue worse.

Capture a representative interval before restart or scale-out changes the evidence:

- request latency and throughput by endpoint and status;
- in-flight requests and server queue time;
- thread/virtual-thread and executor state;
- database and HTTP connection pool acquisition time;
- traces for slow and normal requests;
- repeated thread dumps or a wall-clock profile;
- CPU, GC pauses, allocation, heap, and container throttling;
- downstream and database latency;
- deployment, configuration, and traffic changes.

Use one or a few representative instances. A fleet aggregate can hide a single bad zone, node, or version.

Decompose latency into a budget

Create a stage model even if current telemetry does not expose every stage.

total client budget	1000 ms
edge and network	100 ms
server admission and queue	100 ms
authentication	50 ms
application and database	450 ms
downstream calls	200 ms
serialization and response transfer	100 ms

Budgets are not promises that each stage always consumes its maximum. They prevent each dependency from independently taking the full client timeout. Propagate a deadline or compute remaining time before starting expensive work.

When a stage has already consumed most of the budget, a retry that cannot finish before the deadline is waste. Timeouts should cover connection acquisition, connect, TLS, request write, response headers, body read, database statement, lock acquisition where possible, and total operation.

Queueing is often the hidden latency

Execution may remain fast while requests wait for a server worker, executor, database connection, or downstream connection. Measure queue or acquisition time separately from operation time.

Little's Law provides a useful check:

```
concurrency ~= arrival rate * time in system  
500 requests/s * 0.4 s ~= 200 concurrent requests
```

If the service allows only 100 useful concurrent operations, queues must grow or work must be rejected at that load. An unbounded queue can keep CPU moderate while latency and memory climb.

Thread dumps may show many threads waiting for the same pool or semaphore. Pool metrics should include active, idle, pending, maximum, acquisition duration, and timeout count.

Use traces as a map, not unquestioned truth

A distributed trace can show which child span dominates a slow request and whether calls are sequential or parallel.

GET /checkout	820 ms
auth	20 ms
inventory	180 ms
pricing	170 ms
payment precheck	390 ms
serialize	25 ms

Check for missing spans, incorrect asynchronous context propagation, and sampling bias. Tail-based sampling can retain slow traces, while head sampling may miss rare failures. Instrumentation may measure an HTTP call but not time waiting to acquire a connection unless the client exposes it.

Do not add high-cardinality tags such as raw URL, user ID, or SQL text to metrics. Use route templates and bounded dependency names. Traces can carry more detail under retention and privacy controls.

A latency evidence matrix

Different evidence patterns point toward different next checks:

```
high latency + high CPU + runnable queue
  -> CPU saturation, algorithm, allocation, throttling

high latency + low CPU + many pool waiters
  -> database/HTTP connection or executor bottleneck

high client time + normal edge/server time
  -> network transfer, client retries, client consumption

periodic fleet-wide spikes
  -> GC, scheduled work, expiry wave, checkpoint, rotation

one instance or zone only
  -> skew, routing, host, version, or local dependency
```

This is triage, not proof. For example, low CPU can coexist with a database bottleneck or with container throttling that reports CPU in an unexpected unit. Each branch should lead to direct evidence.

Create a timeline that aligns latency, in-flight work, pool waits, CPU/throttling, GC/safepoints, database waits, downstream errors, and deployment events. Correlation does not establish causation, but it rules out many disconnected explanations.

Safepoints and JVM-wide pauses

Application threads can stop for JVM operations other than the most visible GC pause. Class redefinition, biased-lock-related historical behavior on older JDKs, deoptimization, thread dumps, heap inspection, and other VM operations can involve safepoints or handshakes depending on JDK behavior.

JFR provides safepoint and VM-operation evidence. If many unrelated requests pause together on one instance while dependencies remain healthy, inspect JVM-wide events, operating-system scheduling, container throttling, and host pauses.

Do not enable excessively verbose diagnostics during the incident without understanding their effect. Some heap operations and thread inspection commands can pause or heavily load a large JVM.

Asynchronous work and context gaps

An HTTP span may end when work is queued, while the real business completion occurs later. Conversely, a request can wait for a `CompletableFuture` whose child work lost tracing context, leaving a large unexplained parent duration.

Name and instrument executors, propagate observation context deliberately, and record queue time separately from task execution. Verify that timeouts cancel or detach work appropriately; timed-out background work can continue consuming downstream capacity and slow later requests.

For messaging-backed APIs that return accepted status, measure both acknowledgement latency and end-to-end business completion age. Optimizing the HTTP response while the durable queue grows does not improve the user's actual outcome.

Server thread and executor saturation

For platform-thread servers, compare busy threads, maximum threads, accept queue, and request queue. More threads can increase throughput for blocking I/O until connection pools or downstream capacity saturate; beyond that they add memory and scheduling overhead.

Executors should be named and instrumented. A shared executor lets slow optional work delay unrelated requests. Bounded bulkheads separate critical paths.

Virtual threads reduce the cost of blocking threads and can remove a traditional worker-thread bottleneck, but they do not remove queueing for database connections, remote quotas, CPU, or synchronized critical sections. Measure in-flight work and scarce-resource waits even when thread count is large.

Repeated thread dumps or a wall-clock profile reveal common waiting stacks. A CPU profile alone can miss latency dominated by blocking.

Database connection acquisition

An endpoint may report slow SQL even when the statement itself is fast because it first waited for a connection. Separate:

```
pool acquisition -> network/connect if needed -> database execution
-> result transfer -> ORM mapping -> connection return
```

Pool exhaustion can result from long transactions, leaked connections, slow SQL, remote calls inside transactions, or an oversized application fleet whose total pool connections exceed database capacity.

Increasing every instance's pool can worsen database contention. Calculate total possible connections across replicas, jobs, admin tools, and failover headroom. Use acquisition timeouts shorter than the request deadline and expose pending borrowers.

Slow SQL and query plans

Capture statement fingerprints and database execution plans using approved tooling. Look for table scans, poor join order, cardinality-estimation errors, sorts or spills, lock waits, stale statistics, parameter-sensitive plans, and increased rows returned.

Application traces should distinguish SQL execution from ORM hydration and lazy follow-up queries. N+1 behavior can create hundreds of individually fast statements that make the endpoint slow.

```
1 query for 100 orders
+ 100 queries for order items
= 101 round trips despite no single "slow query"
```

Compare rows examined, rows returned, round trips, and database CPU or I/O. An index can accelerate reads while increasing write cost and storage. Fix the access path, fetch plan, or data model based on workload evidence.

Transaction scope and lock waiting

Long transactions hold connections, locks, and snapshots. A Java method may perform an HTTP call or expensive transformation inside a transaction, extending database resource ownership while no SQL executes.

Trace transaction boundaries and inspect database lock waits. Deadlocks normally produce errors after victim selection; blocking chains produce latency. Identify the head blocker, its transaction age, statement, and application operation.

Shorten transactions, establish consistent update order, index predicates that select rows for modification, and avoid user or network waiting inside a transaction. A larger lock timeout can increase tail latency without improving completion.

Downstream calls and deadline propagation

Sequential remote calls add latency. Parallel calls reduce critical-path time only when independent and when downstreams can accept the extra concurrency.

```
sequential: A 150 ms + B 180 ms + C 120 ms ~= 450 ms
parallel:   max(A, B, C) + coordination ~= 180 ms
```

Fan-out also lowers combined reliability and increases connection demand. One request calling fifty shards can have poor tail latency even when each shard is usually fast.

Set per-attempt timeouts from the remaining deadline. Retry only transient and idempotent operations with a small budget and jitter. Record attempts, not only final logical calls. Hedged requests can reduce tail latency for carefully chosen idempotent reads, but they intentionally add load and require cancellation of losers.

DNS, connect, TLS, and connection reuse

HTTP latency is not just server processing. Connection pool starvation, DNS delay, TCP connect, TLS handshake, proxy traversal, stale connections, and response-body reading can dominate.

Reuse a configured `HttpClient` rather than constructing one per request; client instances manage connection pools that are typically not shared. Monitor connection acquisition and pool limits. Validate DNS caching and failover behavior for the deployed JDK and platform.

Intermittent stale-connection failures can trigger retries and make latency bimodal. Compare fresh connection versus reused connection spans and inspect load-balancer idle timeouts. Keep-alive settings must align across client, proxy, and server.

CPU, throttling, and garbage collection

When CPU saturates, runnable work queues and latency rises nonlinearly. Check CPU per request, container throttling, JIT warm-up, and the high-CPU workflow from Chapter 59.

Stop-the-world GC pauses suspend application progress. Concurrent GC phases consume CPU and can slow requests without one obvious long pause. Correlate JFR or GC events with latency timestamps, allocation rate, live-set occupancy, and safepoint duration.

A request allocating large temporary graphs may cause system-wide GC cost even when its own trace attributes most time to application code. Compare allocation by endpoint or stack where practical.

Locks and synchronized bottlenecks

A small synchronized region can serialize requests across otherwise independent threads. Thread dumps show many blocked threads and one owner; lock profiles quantify contention.

Common causes include a synchronized cache loader, single shared formatter in legacy code, coarse registry lock, class initialization, and connection or token refresh guarded by one monitor.

Reduce critical-section work, use concurrent structures with atomic operations, partition state, or assign ownership. Replacing a lock with complicated lock-free code is not automatically safer or faster. First establish contention and required invariants.

Serialization, payloads, and slow clients

Large or deeply nested responses consume query time, heap, CPU, network bandwidth, and client parsing. Measure response-size distributions and serialization duration. Apply pagination, field selection, compression based on measured tradeoffs, and hard size limits.

Streaming reduces the need to buffer a complete response but holds a connection and server resources while the client reads. Slow clients can therefore reduce available connections and increase direct-buffer pressure even after application computation finishes.

Check proxy and server write time, not only controller duration. Do not log complete payloads on the slow path; it increases CPU, allocation, I/O, and data exposure.

Cache behavior and latency

A cache miss makes the source path visible. A deployment, eviction storm, key-version change, or regional failover can lower hit rate and overload the database. Correlate latency with hit/miss, load concurrency, eviction, and stale-serving mode.

The average may remain healthy while one hot missing key creates a stampede. Use request coalescing, TTL jitter, bounded loaders, and stale-while-revalidate where correctness permits.

Caching can also add latency when the distributed cache is slow and the application waits before falling back. Use short cache timeouts and protect the authoritative store from unbounded miss traffic.

Periodic and multimodal latency

Latency spikes at regular intervals suggest scheduled jobs, cache expiry waves, log rotation, certificate refresh, full GC, database checkpoints, statistics collection, autoscaling, or synchronized background refresh.

A bimodal distribution often means two paths: cache hit versus miss, local versus remote region, reused versus fresh connection, old versus new version, or healthy versus retry. Segment before averaging the modes together.

Use heat maps or histograms over time, not only one percentile line. A percentile can hide the shape and may change because traffic mix changed.

Load testing without coordinated omission

A closed-loop load generator that waits for each slow response before sending the next request reduces offered load during the slowdown. It can underreport queueing and tail latency. An open or arrival-rate-controlled test better represents independent user arrivals, but it must be bounded to avoid accidental damage.

Record the intended send time so latency includes time the system could not accept work. Test realistic payload, tenant skew, cache state, connection reuse, authentication, and data volume. Include cold start, dependency slowdown, and recovery.

The objective is not maximum requests per second at any latency. Test the highest sustainable throughput that meets the latency, error, and correctness SLO.

Mitigation and root-cause correction

Mitigation may revert a deployment, disable optional fan-out, pause batch work, shed load, reduce retries, serve stale data, isolate a tenant, repair an index, or add validated capacity. Avoid broad timeout increases and unlimited queues.

Verify the permanent fix with the same traffic shape and failure condition. Compare client and server percentiles, stage duration, queue and acquisition time, throughput, errors, CPU, allocation, database load, and downstream calls per request.

Add observability at the stage that was invisible. A postmortem should explain why the system admitted more work than it could complete, why the latency budget did not contain the dependency, or why a rollout guard missed the regression.

Common senior-level traps

"The controller took 100 ms, so the network is slow." Verify edge, queue, transfer, retries, and client timing before assigning ownership.

"The SQL is fast." Connection wait, N+1 round trips, result transfer, and ORM mapping may dominate.

"Increase the thread and connection pools." This can transfer contention to the database and worsen tail latency.

"Raise the timeout." Longer waits consume capacity and postpone failure; fix the budget and bottleneck.

"The trace proves the full path." Missing spans, sampling, and unmeasured pool waits can leave gaps.

"Virtual threads eliminate server latency." They reduce thread cost, not downstream, connection, CPU, lock, or queue limits.

A strong interview answer

I define the affected route and compare client and server p50, p95, p99, timeouts, throughput, response size, version, tenant, and region. I preserve traces, repeated thread state or a wall profile, pool metrics, CPU/GC, database evidence, and downstream timing before disruptive mitigation. I decompose the deadline into edge, admission queue, execution, connection acquisition, SQL, remote calls, serialization, and transfer. I look for saturation and queueing before increasing pools, distinguish statement time from connection and ORM time, and account for retries and fan-out. I correlate periodic or bimodal latency with cache, connection, GC, deployment, and background events. The fix is verified under realistic arrival-rate load using end-to-end percentiles, stage durations, resource saturation, error rate, and correctness.

Chapter summary

Latency diagnosis is elapsed-time accounting across queues and dependencies. Traces provide a map, metrics reveal population behavior, thread and pool evidence explain waiting, and database or network tools validate the remote stages. The strongest design keeps work bounded by deadlines and rejects overload before queues turn it into tail latency.

Revision Notes

- Define the normalized endpoint, percentile, time window, and affected dimensions.
- Client-observed latency includes more stages than application server timing.
- Averages hide tail behavior; interpret percentiles with sample count and window.
- Preserve traces, pool state, thread evidence, CPU/GC, and dependency timing before restart.
- Build an explicit latency budget and propagate the remaining deadline.
- Measure queue and connection acquisition separately from execution.
- Little's Law connects arrival rate, time in system, and concurrency.
- Unbounded queues convert overload into latency and memory growth.
- Traces can miss spans, retries, and acquisition waits.
- Use wall profiles for waiting-heavy latency and CPU profiles for computation.
- Virtual threads do not remove database, remote, CPU, or lock limits.
- Calculate total database connections across every application replica and job.
- N+1 produces many fast queries and one slow request.
- Keep transactions short and avoid remote calls while holding them.
- Fan-out reduces critical-path time but increases load and tail-risk exposure.
- Reuse HTTP clients and measure DNS, connect, TLS, pool, and body-read time.
- Correlate GC and container throttling with latency timestamps.
- Measure response size, serialization, write time, and slow-client behavior.
- Segment cache hits/misses and multimodal paths.
- Load-test realistic arrival rates, skew, cold state, failures, and recovery.